```tsx
"use client";

import React, {
  forwardRef,
  useRef,
  useState,
  useEffect,
  useCallback,
  useMemo,
} from "react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🚀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🎛️ LOCAL TYPE ISOLATION GATEWAY
export interface SliderProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "value" | "defaultValue" | "type"
> {
  label?: string;
  error?: string;
  min?: number;
  max?: number;
  step?: number;
  value?: number | string;
  defaultValue?: number | string;
  stepsArray?: string[];
  onChange?: (value: number | string) => void;
  showValue?: boolean;
  orientation?: "horizontal" | "vertical";
}

export const Slider = forwardRef<HTMLInputElement, SliderProps>(
  (
    {
      label,
      error,
      min = 0,
      max = 100,
      step = 1,
      value,
      defaultValue,
      stepsArray,
      onChange,
      className,
      disabled,
```

```
    showValue = true,
    orientation = "horizontal",
    ...props
  },
  ref,
) => {
  const sliderRef = useRef<HTMLDivElement>(null);
  const thumbRef = useRef<HTMLDivElement>(null);
  const trackRef = useRef<HTMLDivElement>(null);
  const isDraggingRef = useRef(false);
  const startXRef = useRef(0);
  const startValueRef = useRef(0);

  const isEnumMode = stepsArray && stepsArray.length > 0;
  const enumMin = 0;
  const enumMax = isEnumMode ? stepsArray.length - 1 : max;
  const enumStep = isEnumMode ? 1 : step;
  const effectiveMin = isEnumMode ? enumMin : min;
  const effectiveMax = isEnumMode ? enumMax : max;
  const effectiveStep = isEnumMode ? enumStep : step;

  const [internalValue, setInternalValue] = useState<number>(() => {
    if (value !== undefined) {
      if (isEnumMode && typeof value === "string") {
        return stepsArray!.indexOf(value);
      }
      return typeof value === "number"
        ? value
        : parseFloat(value as string) || effectiveMin;
    }
    if (defaultValue !== undefined) {
      if (isEnumMode && typeof defaultValue === "string") {
        return stepsArray!.indexOf(defaultValue);
      }
      return typeof defaultValue === "number"
        ? defaultValue
        : parseFloat(defaultValue as string) || effectiveMin;
    }
    return effectiveMin;
  });

  const [focused, setFocused] = useState(false);
  const [hovered, setHovered] = useState(false);

  const clampedValue = useMemo(() => {
    let val = internalValue;
    if (val < effectiveMin) val = effectiveMin;
    if (val > effectiveMax) val = effectiveMax;
    const stepped =
      Math.round((val - effectiveMin) / effectiveStep) * effectiveStep +
```

```
    effectiveMin;
  return Math.min(Math.max(stepped, effectiveMin), effectiveMax);
}, [internalValue, effectiveMin, effectiveMax, effectiveStep]);

const percentage = useMemo(() => {
  if (effectiveMax === effectiveMin) return 0;
  return (
    ((clampedValue - effectiveMin) / (effectiveMax - effectiveMin)) * 100
  );
}, [clampedValue, effectiveMin, effectiveMax]);

const displayValue = useMemo(() => {
  if (isEnumMode) {
    return stepsArray[clampedValue] ?? "";
  }
  return clampedValue;
}, [isEnumMode, stepsArray, clampedValue]);

const isActuallyDisabled = disabled ?? false;
const hasError = !!error;

useEffect(() => {
  if (value !== undefined) {
    if (isEnumMode && typeof value === "string") {
      const idx = stepsArray!.indexOf(value);
      if (idx !== -1) setInternalValue(idx);
    } else {
      const numVal =
        typeof value === "number" ? value : parseFloat(value as string);
      if (!isNaN(numVal)) setInternalValue(numVal);
    }
  }
}, [value, isEnumMode, stepsArray]);

const notifyChange = useCallback(
  (newValue: number) => {
    const finalValue = isEnumMode ? stepsArray[newValue] : newValue;
    onChange?.(finalValue);
  },
  [isEnumMode, stepsArray, onChange],
);

const updateValueFromPosition = useCallback(
  (clientX: number, clientY: number) => {
    if (!trackRef.current) return;

    const rect = trackRef.current.getBoundingClientRect();
    let newPercentage: number;

    if (orientation === "horizontal") {
```

```
      newPercentage = ((clientX - rect.left) / rect.width) * 100;
    } else {
      newPercentage = ((rect.bottom - clientY) / rect.height) * 100;
    }

    newPercentage = Math.max(0, Math.min(100, newPercentage));
    const newValue =
      effectiveMin + (newPercentage / 100) * (effectiveMax - effectiveMin);
    const steppedValue =
      Math.round((newValue - effectiveMin) / effectiveStep) *
        effectiveStep +
      effectiveMin;
    const clamped = Math.min(
      Math.max(steppedValue, effectiveMin),
      effectiveMax,
    );

    setInternalValue(clamped);
    notifyChange(clamped);
  },
  [effectiveMin, effectiveMax, effectiveStep, orientation, notifyChange],
);

const handleMouseDown = useCallback(
  (
    e: React.MouseEvent<HTMLDivElement> | React.TouchEvent<HTMLDivElement>,
  ) => {
    if (isActuallyDisabled) return;

    isDraggingRef.current = true;
    const clientX = "touches" in e ? e.touches[0].clientX : e.clientX;
    const clientY = "touches" in e ? e.touches[0].clientY : e.clientY;
    startXRef.current = clientX;
    startValueRef.current = clampedValue;

    e.preventDefault();
    updateValueFromPosition(clientX, clientY);
  },
  [isActuallyDisabled, clampedValue, updateValueFromPosition],
);

const handleMouseMove = useCallback(
  (e: MouseEvent | TouchEvent) => {
    if (!isDraggingRef.current || isActuallyDisabled) return;

    const clientX = "touches" in e ? e.touches[0].clientX : e.clientX;
    const clientY = "touches" in e ? e.touches[0].clientY : e.clientY;
    updateValueFromPosition(clientX, clientY);
  },
  [isActuallyDisabled, updateValueFromPosition],
```

```
);

const handleMouseUp = useCallback(() => {
  isDraggingRef.current = false;
}, []);

useEffect(() => {
  if (isDraggingRef.current) {
    document.addEventListener(
      "mousemove",
      handleMouseMove as EventListener,
    );
    document.addEventListener("mouseup", handleMouseUp);
    document.addEventListener(
      "touchmove",
      handleMouseMove as EventListener,
      { passive: false },
    );
    document.addEventListener("touchend", handleMouseUp);
  }

  return () => {
    document.removeEventListener(
      "mousemove",
      handleMouseMove as EventListener,
    );
    document.removeEventListener("mouseup", handleMouseUp);
    document.removeEventListener(
      "touchmove",
      handleMouseMove as EventListener,
    );
    document.removeEventListener("touchend", handleMouseUp);
  };
}, [handleMouseMove, handleMouseUp]);

const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLDivElement>) => {
    if (isActuallyDisabled) return;

    let newValue = clampedValue;
    let shouldUpdate = false;

    switch (e.key) {
      case "ArrowRight":
      case "ArrowUp":
        e.preventDefault();
        newValue = Math.min(clampedValue + effectiveStep, effectiveMax);
        shouldUpdate = true;
        break;
      case "ArrowLeft":
```

```
      case "ArrowDown":
        e.preventDefault();
        newValue = Math.max(clampedValue - effectiveStep, effectiveMin);
        shouldUpdate = true;
        break;
      case "Home":
        e.preventDefault();
        newValue = effectiveMin;
        shouldUpdate = true;
        break;
      case "End":
        e.preventDefault();
        newValue = effectiveMax;
        shouldUpdate = true;
        break;
      case "PageUp":
        e.preventDefault();
        newValue = Math.min(
          clampedValue + effectiveStep * 10,
          effectiveMax,
        );
        shouldUpdate = true;
        break;
      case "PageDown":
        e.preventDefault();
        newValue = Math.max(
          clampedValue - effectiveStep * 10,
          effectiveMin,
        );
        shouldUpdate = true;
        break;
    }

    if (shouldUpdate) {
      setInternalValue(newValue);
      notifyChange(newValue);
    }
  },
  [
    isActuallyDisabled,
    clampedValue,
    effectiveMin,
    effectiveMax,
    effectiveStep,
    notifyChange,
  ],
);

const handleTrackClick = useCallback(
  (e: React.MouseEvent<HTMLDivElement>) => {
```

```
        if (isActuallyDisabled) return;
        if (e.currentTarget === e.target) {
          updateValueFromPosition(e.clientX, e.clientY);
        }
      },
      [isActuallyDisabled, updateValueFromPosition],
    );

    const combinedRef = useCallback(
      (el: HTMLInputElement | null) => {
        if (ref) {
          if (typeof ref === "function") {
            ref(el);
          } else {
            ref.current = el;
          }
        }
      },
      [ref],
    );

    const trackClasses = cn(
      "relative",
      "w-full h-2",
      "rounded-full bg-border",
      "transition-colors duration-200 ease-out",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
      "disabled:pointer-events-none disabled:opacity-50
disabled:cursor-not-allowed",
      "aria-invalid:border-destructive aria-invalid:ring-3
aria-invalid:ring-destructive/20",
      hasError && "border-destructive focus-visible:ring-destructive",
      orientation === "vertical" && "w-2 h-full",
      className,
    );

    const progressClasses = cn(
      "absolute",
      "rounded-full",
      "bg-brand-primary",
      "transition-all duration-150 ease-out",
      orientation === "horizontal"
        ? "h-full left-0 top-0"
        : "w-full bottom-0 left-0",
    );

    const thumbClasses = cn(
      "absolute",
      "w-5 h-5",
```

```
      "rounded-full bg-white",
      "shadow-lg shadow-black/15",
      "border-2 border-brand-primary",
      "transition-all duration-150 ease-out",
      "active:scale-110",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-brand-primary
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
      "hover:scale-110 hover:shadow-xl",
      "disabled:pointer-events-none disabled:opacity-50
disabled:cursor-not-allowed",
      "aria-invalid:border-destructive aria-invalid:ring-3
aria-invalid:ring-destructive/20",
      orientation === "horizontal"
        ? "top-1/2 -translate-y-1/2 -translate-x-1/2"
        : "left-1/2 -translate-x-1/2 -translate-y-1/2",
      focused &&
        "ring-2 ring-brand-primary ring-offset-2 ring-offset-background",
      hovered && "scale-110 shadow-xl",
    );

    const labelClasses = cn(
      "block text-sm font-medium leading-none",
      "transition-colors duration-200 ease-out",
      "text-text-main",
      hasError && "text-destructive/90",
      isActuallyDisabled && "opacity-50",
      "mb-2",
    );

    const valueDisplayClasses = cn(
      "absolute",
      "text-xs font-medium",
      "text-text-main",
      "transition-all duration-150 ease-out",
      "pointer-events-none",
      "white-space-nowrap",
      orientation === "horizontal"
        ? "bottom-full left-1/2 -translate-x-1/2 mb-1.5"
        : "right-full top-1/2 -translate-y-1/2 mr-2",
      isEnumMode && "min-w-[60px] text-center",
    );

    const containerClasses = cn(
      "inline-flex flex-col gap-2",
      orientation === "vertical" && "items-center",
      isActuallyDisabled && "opacity-50 pointer-events-none",
    );

    const errorClasses = cn(
      "text-sm",
```

```
  "text-destructive/90",
  "transition-colors duration-200 ease-out",
  "mt-1",
);

return (
  <div
    className={containerClasses}
    role="group"
    aria-invalid={hasError}
    aria-disabled={isActuallyDisabled}
  >
    {label && <label className={labelClasses}>{label}</label>}

    <div
      ref={sliderRef}
      className={cn(
        "relative",
        orientation === "horizontal" ? "w-full" : "h-64",
      )}
      onMouseDown={handleMouseDown}
      onTouchStart={
        handleMouseDown as React.TouchEventHandler<HTMLDivElement>
      }
      onKeyDown={handleKeyDown}
      onMouseEnter={() => setHovered(true)}
      onMouseLeave={() => setHovered(false)}
      onFocus={() => setFocused(true)}
      onBlur={() => setFocused(false)}
      tabIndex={isActuallyDisabled ? -1 : 0}
      role="slider"
      aria-label={label}
      aria-valuemin={effectiveMin}
      aria-valuemax={effectiveMax}
      aria-valuenow={clampedValue}
      aria-valuetext={String(displayValue)}
      aria-orientation={orientation}
      aria-disabled={isActuallyDisabled}
      aria-invalid={hasError ? "true" : "false"}
      aria-required={props.required}
    >
      <div
        ref={trackRef}
        className={trackClasses}
        onClick={handleTrackClick}
        role="presentation"
        aria-hidden="true"
      >
        <div
          className={progressClasses}
```

```
      style={
        orientation === "horizontal"
          ? { width: `${percentage}%` }
          : { height: `${percentage}%` }
      }
      role="presentation"
      aria-hidden="true"
    />
    <div
      ref={thumbRef}
      className={thumbClasses}
      style={
        orientation === "horizontal"
          ? { left: `${percentage}%` }
          : { bottom: `${percentage}%` }
      }
      role="presentation"
      aria-hidden="true"
    />
  </div>

  {showValue && (
    <div
      className={valueDisplayClasses}
      style={
        orientation === "horizontal"
          ? { left: `${percentage}%` }
          : { bottom: `${percentage}%` }
      }
      aria-hidden="true"
    >
      {displayValue}
    </div>
  )}
</div>

{error && (
  <p className={errorClasses} role="alert" aria-live="polite">
    {error}
  </p>
)}

<input
  ref={combinedRef}
  type="hidden"
  value={isEnumMode ? displayValue : clampedValue}
  disabled={isActuallyDisabled}
  required={props.required}
  name={props.name}
/>
```

```
      </div>
    );
  },
);

Slider.displayName = "Slider";
```